

Depuración de Errores y Detección de Vulnerabilidades con IA

Ejercicio de Clase · Ejercicio 1 de 3 · Bloques 2.1 — 2.2 — 2.3 — 2.6

Nombre del alumno/a	Angel Echenique
Fecha	04/042026
🔗 Enlace GitHub	

📌 **Instrucciones:** Responde con lo que hemos visto en clase. Puedes consultar tus apuntes. No hay que escribir código, solo demostrar que entendiste!

BLOQUE 1 — ¿Por qué usar IA para depuración? (2.1)

En clase vimos 5 ventajas clave de usar IA para depurar código. Escríbelas y explica brevemente para qué sirve cada una:

Nº	Nombre de la ventaja	¿Para qué sirve concretamente?
1	Analiza errores rápidamente	Sirve para leer mensajes extensos y confusos, interpretándolos en un lenguaje claro para no quedarnos "atascados".
2	Explica trazas complejas	Sirve para identificar el origen real del fallo, explicar cada parte de la traza y señalar el módulo responsable.
3	Sugiere soluciones paso a paso	Sirve para proponer cómo solucionar el error, indicando qué línea corregir o qué validación falta.
4	Identifica malas prácticas	Sirve para detectar patrones peligrosos como variables sin inicializar o validaciones insuficientes, ayudando a mejorar nuestro criterio técnico.
5	Reduce tiempo de debugging	Sirve para avanzar más rápido, dedicando más tiempo a comprender el código que a pelearnos con él.

1.2 — Depuración tradicional vs. Depuración con IA

Completa la siguiente tabla indicando cómo cambia cada aspecto del proceso de depuración al usar IA:

Aspecto	Depuración Tradicional	Depuración con IA
Velocidad de análisis	Es un proceso lento, manual y de prueba/error constante.	Ofrece respuestas rápidas, automatizadas y diagnósticos mas preciso.
Comprensión de errores complejos	Es difícil de interpretar con información dispersa.	Ofrece explicaciones claras, adaptadas y centralizadas en una sola conversación.
Disponibilidad del asistente	Experiencia previa o buscar en foros.	Accesibilidad para todo los niveles con aprendizaje guiado.
Personalización de la solución	Encontramos soluciones genéricas.	Obtenemos soluciones personalizadas.
Curva de aprendizaje	Curva de aprendizaje más pronunciada.	Aprendizaje guiado y progresivo.

BLOQUE 2 — Tipos de errores que la IA detecta bien (2.2)

La IA detecta 4 tipos principales de errores. Clasifica cada ejemplo en su categoría: L (Lógico), E (Ejecución), A (Asincronía) o D (Diseño), y explica brevemente por qué:

L/ E/ A/ D	Ejemplo de código / situación	¿Por qué es de ese tipo?
L	<i>Condición if edad >= 18 pero la segunda rama elif edad < 18 nunca se ejecuta cuando edad == 18</i>	Es un error de condiciones incorrectas o lógica mal definida.
A	<i>Función async que llama a fetch() sin usar await, recibiendo [object Promise]</i>	Promesas sin await Callbacks mal encadenados Race conditions simples.
E	<i>División total / len(notas) que falla con ZeroDivisionError cuando la lista está vacía</i>	Es una excepción no controlada (ZeroDivisionError) que ocurre mientras el programa corre.
D	<i>Función de 200 líneas con lógica de 3 responsabilidades distintas mezcladas</i>	Es un problema de funciones demasiado grandes y falta de modularidad.
E	<i>Variable 'descuento' definida solo dentro del if, usada fuera →NameError</i>	Genera un NameError al intentar usar una variable no definida en ese ámbito.
A	<i>Callbacks encadenados sin manejo de errores que provoca race condition</i>	Pertenece a la categoría de callbacks mal encadenados y race conditions.
D	<i>Módulo A importa al módulo B y el módulo B importa al módulo A →ImportError</i>	Es un error de dependencias circulares.
E	<i>suma('5', 10) →TypeError: can only concatenate str (not int) to str</i>	Es un error por tipos incorrectos (mezclar texto con números).

BLOQUE 3 — Cómo pedir a la IA que depure un error (2.3)

Un prompt profesional para depurar debe incluir 5 elementos clave. Escríbelos y explica para qué sirve cada uno:

Nº	Elemento del prompt	¿Para qué sirve?
1	Código completo o fragmento relevante	Sirve para que la IA vea la función, clase o bloque exacto donde ocurre el error.
2	Error exacto (mensaje y traza)	Sirve para identificar la causa raíz con precisión indicando la línea afectada y el módulo.
3	Contexto (qué debería hacer)	Sirve para explicar el comportamiento esperado y los datos de entrada; sin esto, la IA solo puede adivinar.
4	Entorno (lenguaje, framework, versión)	Sirve para evitar sugerencias incompatibles, ya que muchos errores dependen de las versiones o bases de datos usadas.
5	Qué tipo de ayuda necesitas	Sirve para mejorar la calidad de la respuesta, indicando si quieres una explicación, una solución segura, o detectar vulnerabilidades.

3.2 — Mejora estos prompts

Reescribe cada prompt deficiente para convertirlo en un prompt profesional y efectivo:

✗ Prompt deficiente	✓ Prompt mejorado
"Tengo un error, ¿qué puede ser?"	Analiza este error [Pegamos el error dado] en mi función [Pegamos el código]. Esperaba que hiciera [X], dime las causas del error y cómo arreglarlo.
"Mi código no funciona."	Estoy usando Python 3.10. Mi código [el código en cuestión] debería devolver [X], pero no hace nada. Explícame la causa raíz.
"Arréglame esto."	Haz un análisis de causa raíz de este error en mi archivo de kindred.py y sugiere una solución robusta siguiendo buenas prácticas.

BLOQUE 4 — Detección de vulnerabilidades con IA (2.6)

Une cada vulnerabilidad con su descripción correcta y escribe el patrón de código peligroso que la IA detecta:

Vulnerabilidad	Descripción	Patrón peligroso que detecta la IA
SQL Injection	<i>El atacante engaña al servidor para ejecutar consultas no autorizadas en la BD.</i>	<i>Concatenación directa de strings en consultas o parámetros no sanitizados.</i>
XSS	<i>Permite a un atacante ejecutar scripts maliciosos en el navegador de la víctima.</i>	<i>Contenido no escapado en plantillas o uso inseguro de innerHTML.</i>
CSRF	<i>Formularios que ejecutan acciones sensibles sin verificar la identidad del solicitante</i>	<i>Formularios sin tokens CSRF o endpoints sensibles sin protección.</i>
Exposición de datos	<i>Envío o almacenamiento de información sensible sin cifrar</i>	<i>Envío de contraseñas en claro, respuestas con API keys o tokens.</i>
Dependencias inseguras	<i>Uso de librerías con CVEs conocidos sin actualizar</i>	<i>Librerías obsoletas o versiones con CVEs conocidos.</i>
Errores de autenticación	<i>Endpoints que permiten acceso sin verificar permisos ni identidad</i>	<i>Endpoints sin protección, roles mal definidos o tokens no verificados.</i>

4.2 — Verdadero o Falso

Marca V (Verdadero) o F (Falso). Justifica brevemente las que sean Falsas:

V/F	Afirmación
V	La IA puede detectar vulnerabilidades XSS revisando el uso de innerHTML sin sanitizar.
F	Para que la IA haga una auditoría de seguridad solo necesita ver el mensaje de error. JUSTIFICACION: La IA necesita ver el código completo, el contexto funcional y el entorno tecnológico.
V	OWASP Top 10 es el estándar más común para auditorías de seguridad web con IA.
F	Si el código usa parámetros preparados en SQL, ya no existe riesgo de SQL Injection. JUSTIFICACION: La IA también debe revisar si hay un "uso incorrecto de ORMs" o si existen "consultas construidas dinámicamente sin validación".
V	La IA puede revisar dependencias obsoletas comparando versiones con CVEs conocidos.
F	La IA reemplaza completamente a un profesional pentester en auditorías de seguridad. JUSTIFICACION: La IA funciona como un asistente técnico y analista automatizado, pero no reemplaza a los profesionales; permite pensar como ellos y reforzar la seguridad en etapas tempranas. Pero seguimos siendo nosotros lo que revisamos, agregamos o quitamos test.
V	Un endpoint sin autenticación que devuelve datos de usuarios es una vulnerabilidad crítica.

BLOQUE 5 — Pregunta de síntesis

Un compañero/a dice: "La IA me da el código correcto, así que no necesito entender el error ni la causa raíz; solo copio y pego la solución." ¿Estás de acuerdo? Razona tu respuesta con al menos dos argumentos concretos apoyándote en los 4 bloques de este ejercicio:

Argumento 1: Copiar y pegar evita que comprendas el error. Como vimos, la IA no solo corrige, sino que explica la causa raíz para que aprendas a evitar el error en el futuro, desarrolles pensamiento crítico y mejores tu autonomía como programador.

Argumento 2: Comprender la causa raíz (Root Cause Analysis) es mucho más importante que aplicar un parche rápido. La IA nos ayuda a entender el "qué" provoca el error y las condiciones que lo reproducen, lo cual es vital para implementar medidas preventivas y un diseño más seguro, en lugar de solo apagar el incendio temporalmente.